Solution: Merge Similar Items

Problem Statement

Given two 2D arrays of item-value pairs, named `items1` and `items2`.

- **item[i] = [value$_i$, weight$_i$]**, where each value$_i$ in these arrays is unique within its own array and is paired with a weight$_i$.

Combine these arrays such that if an item appears in both, its values are summed up. The final merged array should be sorted based on the value$_i$.

Examples

1. **Example 1**:
    - **Input**: `items1 = [[1,2],[4,3]]`, `items2 = [[2,1],[4,3],[3,4]]`
    - **Expected Output**: `[[1,2],[2,1],[3,4],[4,6]]`
    - **Justification**: Item 1 has value 2 in `items1` and doesn't exist in `items2`, item 2 has value 1 in `items2`, item 4 is summed up.
2. **Example 2**:
    - **Input**: `items1 = [[5,5]]`, `items2 = [[5,10]]`
    - **Expected Output**: `[[5,15]]`
    - **Justification**: Item 5 exists in both arrays, so their values are summed.
3. **Example 3**:
    - **Input**: `items1 = [[1,1],[2,2]]`, `items2 = [[3,3]]`
    - **Expected Output**: `[[1,1],[2,2],[3,3]]`
    - **Justification**: All items are unique across `items1` and `items2`, so they remain unchanged.

Solution

To solve this problem, the approach involves creating a map (or dictionary) to efficiently track and combine the quantities of items. Start by iterating through each list. For each pair in these lists, check if the item (the first element of the pair) already exists in the map. If it does, update its quantity by adding the new quantity. If it doesn't exist, add the item and its quantity to the map. Once both lists are processed, you will have a map with each item and its total quantity. Finally, convert this map into a list of pairs, sorted by the item numbers. This list represents the merged and sorted items with their cumulative quantities.

Step-by-step algorithm

1. **Merging Lists:** Start by merging both lists into a single list. This step doesn't combine the values of similar items yet; it just creates a single list containing all item-value pairs.
2. **Using an Ordered Set:** Create an ordered set (like TreeSet or TreeMap in Java or std::set in C++) to store the final item-value pairs. The key of the set will be the item identifier, and the value will be the sum of values for that item.
3. **Populating the Set:** Iterate through the merged list. For each item-value pair, check if the item is already in the set. If it is, update the value by adding the new value to the existing one. If it's not, insert the new item-value pair into the set.
4. **Creating the Result List:** Finally, iterate through the ordered set and construct the result list from the elements of the set.

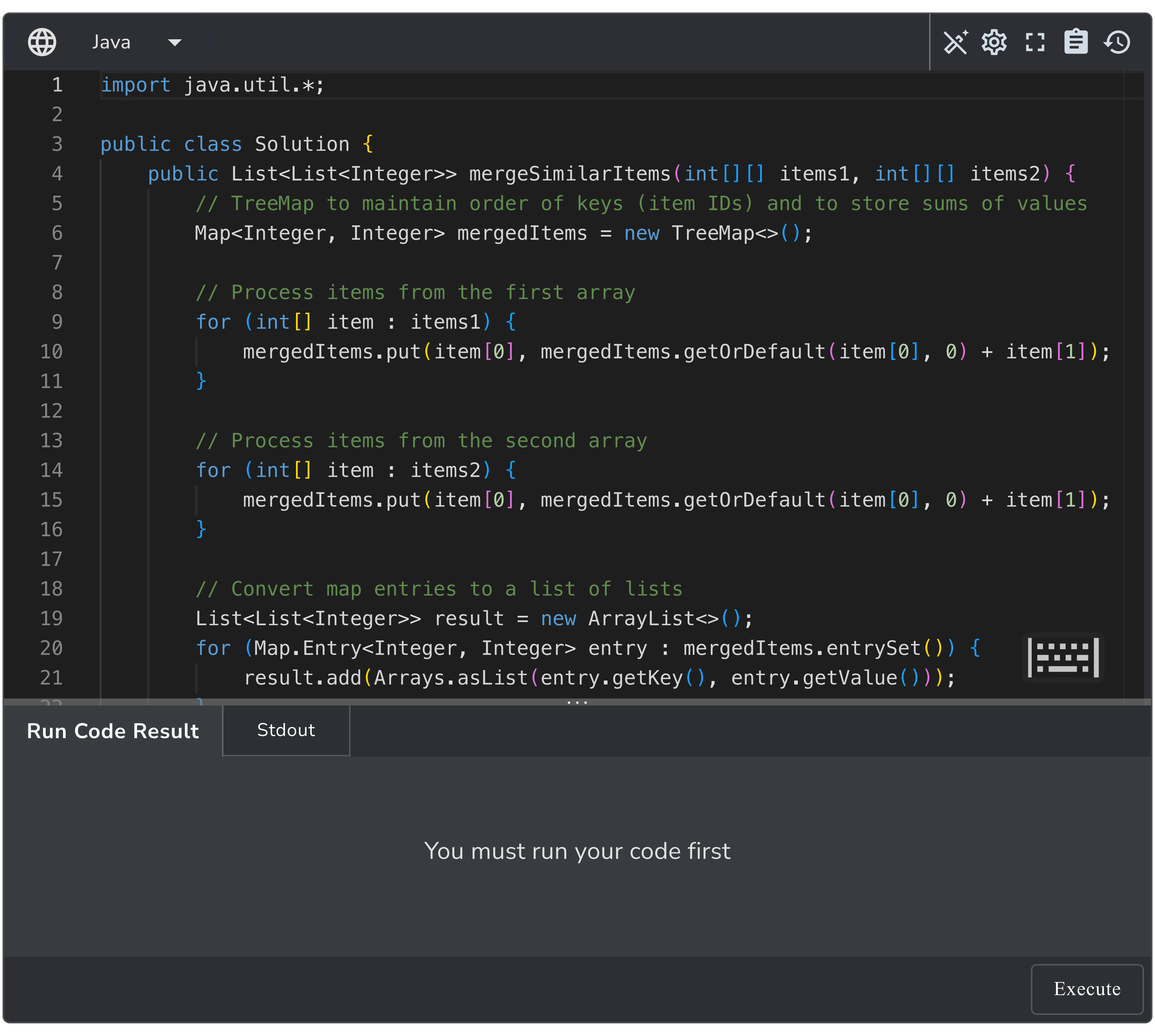
This approach ensures that each item is processed only once, making it efficient. The use of an ordered set guarantees that the final list is sorted by the item identifiers, as required.

Algorithm Walkthrough for Example 1

Let's walk through the algorithm using Example 1, where `items1 = [[1,2],[4,3]]` and `items2 = [[2,1],[4,3],[3,4]]`.

1. **Initialization:** Create an empty map (TreeMap in Java) to store the sum of values for each item. This map will help in merging and summing the item values.
2. **Processing** `items1`:
    - Process item `[1,2]`: Since item `1` is not in the map, add it with the value `2`.
    - Process item `[4,3]`: Since item `4` is not in the map, add it with the value `3`.
    - After processing `items1`, the map looks like `{1=2, 4=3}`.
3. **Processing** `items2`:
    - Process item `[2,1]`: Since item `2` is not in the map, add it with the value `1`.
    - Process item `[4,3]`: Item `4` already exists in the map with value `3`, so update it to `3 + 3 = 6`.
    - Process item `[3,4]`: Since item `3` is not in the map, add it with the value `4`.
    - After processing `items2`, the map looks like `{1=2, 2=1, 3=4, 4=6}`.
4. **Creating Sorted Result:**
    - Convert the map entries into a list. The TreeMap ensures that entries are sorted based on the item identifiers.
    - The resulting list from the map entries is `[[1,2], [2,1], [3,4], [4,6]]`.
5. **Final Output:**
    - Return the sorted list `[[1,2], [2,1], [3,4], [4,6]]`.

Code



```java
import java.util.*;

public class Solution {
    public List<List<Integer>> mergeSimilarItems(int[][] items1, int[][] items2) {
        // TreeMap to maintain order of keys (item IDs) and to store sums of values
        Map<Integer, Integer> mergedItems = new TreeMap<>();

        // Process items from the first array
        for (int[] item : items1) {
            mergedItems.put(item[0], mergedItems.getOrDefault(item[0], 0) + item[1]);
        }

        // Process items from the second array
        for (int[] item : items2) {
            mergedItems.put(item[0], mergedItems.getOrDefault(item[0], 0) + item[1]);
        }

        // Convert map entries to a list of lists
        List<List<Integer>> result = new ArrayList<>();
        for (Map.Entry<Integer, Integer> entry : mergedItems.entrySet()) {
            result.add(Arrays.asList(entry.getKey(), entry.getValue()));
```

You must run your code first

Time and Space Complexity Analysis

- **Time Complexity**: O(N log N), where N is the total number of item-value pairs. This accounts for the merging and sorting operations.
- **Space Complexity**: O(N), for storing the merged items and the result array.